Development: building and evidencing the database application

AD04 FACT FILE

Specification: students should be able to

- Implement a relational database with tables, keys, relationships and referential integrity from the design
- Develop forms with validation, queries including parameter queries and calculated fields, reports and navigation
- Use macros or code to automate processing and improve usability
- Provide annotated evidence of the development of the solution

Key words: table, relationship, referential integrity, validation rule, input mask, lookup, parameter query, calculated field, report, navigation form, macro, annotated screenshot

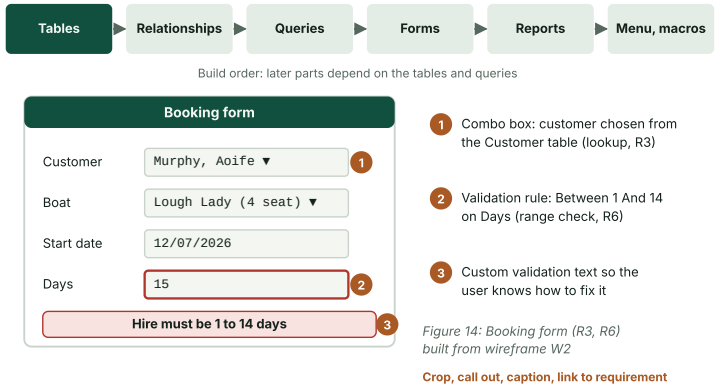


Figure 14: Booking form (R3, R6) built from wireframe W2

Crop, call out, caption, link to requirement

The order for building a database application, and an annotated screenshot of a booking form showing a lookup, a range check and its error message

Implementing the data

Development turns the design into a working application. Tables are created with the data types, sizes, formats and validation from the data dictionary, and each has a primary key. Relationships are created between primary and foreign keys with referential integrity enforced, which prevents orphan records such as a booking for a customer who does not exist. Cascade options must be chosen carefully: cascade delete could remove a customer's entire booking history. Realistic test data should be entered so queries and reports can be checked.

Queries, forms and reports

Queries retrieve and process data. A parameter query prompts the user for criteria when it runs, making one query reusable for any date or surname. A calculated field derives a value from other fields, so derived data such as cost need not be stored. Forms provide a user-friendly way to enter and edit data with validation, combo boxes and clear error messages. Reports format output for printing, often grouped with totals. A navigation menu, macros and VBA code join everything into one easy-to-use application.

Evidence of development

Because the moderator assesses the portfolio rather than the database file, development must be shown through clear, annotated screenshots. Each screenshot should be cropped so it is readable, show the relevant settings (for example design view and the property sheet), and carry a caption with a figure number, the requirement number and the design item it implements. Evidence of iterative development, where a problem was found and fixed, demonstrates understanding and should explain why the change improved the solution.

T Key terms from the mark schemes

Referential integrity
A rule ensuring every foreign key value matches an existing primary key value in the related table, preventing orphan records.

Calculated field
A field in a query, form or report whose value is derived from other fields, such as cost from days and daily rate.

Parameter query
A query that prompts the user to enter a value when it runs and uses that value as its criteria.

Input mask
A pattern that controls the format of data entered into a field, such as a postcode or phone number.

Macro
A stored sequence of actions that automates a task in a database, such as opening a form or printing a report.

Annotated screenshot
A cropped screen capture with labels and a caption explaining what it shows and which requirement and design item it evidences.

C VBA example (Microsoft Access)

Code from a design

```
Private Sub txtDays_AfterUpdate()
    ' R4: calculate the hire cost when the number of days changes
    ' R6: Days must be a whole number from 1 to 14
    Const WEEKLY_DISCOUNT As Double = 0.1
    Dim curRate As Currency
    Dim curCost As Currency

    If IsNull(Me.txtDays) Or Not IsNumeric(Me.txtDays) Then
        MsgBox "Enter the number of days.", vbExclamation, "Booking"
        Exit Sub
    End If
    If Me.txtDays < 1 Or Me.txtDays > 14 Then
        MsgBox "Hire must be 1 to 14 days", vbExclamation, "Booking"
        Me.txtDays.SetFocus
        Exit Sub
    End If

    ' look up the daily rate of the chosen boat
    curRate = Nz(DLookup("DailyRate", "tblBoat", "BoatID = " & Nz(Me.cboBoatID, 0)), 0)
    curCost = curRate * Me.txtDays
    If Me.txtDays >= 7 Then
        curCost = curCost * (1 - WEEKLY_DISCOUNT)    ' weekly discount
    End If
    Me.txtCost = curCost
End Sub
```

Exam focus

Make the sophistication of your solution visible. A complex feature that is not evidenced will not be credited. Show both the setup (design view, SQL, macro or VBA code) and the outcome (the form in use, the query results). Explain why you used a feature, not only what it is. Keep the annotations specific: 'validation rule Between 1 And 14 with custom message meets R6' is far stronger than 'I added validation'. Note and justify any change from your design.

? Check yourself

1. Explain what is meant by referential integrity and why it is important in a booking database. [3]
2. Describe what a parameter query is and give an example from a boat hire system. [3]
3. Explain why the total cost of a booking should be produced by a calculated field rather than stored in the Booking table. [2]

Answers: 1. Referential integrity means every foreign key value must match an existing primary key value in the related table (1) / e.g. a booking cannot be entered for a CustomerID or BoatID that does not exist (no orphan records) (1) / important because bookings must always link to a real customer and boat, so reports, receipts and costs are accurate and the data is consistent; it also prevents a customer with bookings being deleted (if restrict is set) (1). 2. A parameter query prompts the user to enter a value (criterion) when it is run (1) / so one query can be reused for any value instead of creating a separate query for each (1) / example: a query on Booking and Customer with criteria StartDate = [Enter date:] to list all bookings on a date; or search by customer surname (1). 3. The total cost is derived data – it can be calculated from Days × DailyRate whenever needed (1) / storing it would create redundancy and risk inconsistency, e.g. if Days or the daily rate is changed the stored total would be wrong (calculated fields always give the current correct value) (1).